Product Requirements Document

Project Build: Enterprise Policy Assistant with Agentic RAG Using LangGraph

---

1. Project Title

Enterprise Policy Assistant with Agentic RAG Using LangGraph

---

2. Project Duration

4 Hours

Participants are expected to build a working prototype of an enterprise policy assistant using **LangGraph**, **Groq**, **Python**, **LangChain**, and a local policy document corpus.

---

3. Project Context

Enterprises usually maintain many internal policy documents across HR, travel, reimbursement, IT security, data privacy, and AI usage. Employees often struggle to find the correct policy section, understand eligibility rules, and determine whether their scenario is covered.

A normal chatbot may answer from general knowledge and hallucinate policy details. A basic RAG system can retrieve policy content, but it may fail when the question requires:

1. Searching across multiple policies.
2. Deciding whether the retrieved context is relevant.
3. Rewriting weak queries.
4. Asking clarifying questions.
5. Refusing unsupported or speculative answers.
6. Combining multiple retrieved sections into a final answer.
7. Reviewing whether the answer is grounded in policy evidence.

The goal of this project is to build an **Agentic RAG Policy Assistant** using **LangGraph**. The assistant should not behave like a simple one-step RAG chatbot. It should use a graph-based workflow that includes retrieval, grading, conditional branching, parallel retrieval, and grounded answer generation.

---

4. Business Problem

Employees ask policy questions such as:

```
Can I claim meals during same-day business travel?

Can I upload customer data to a public AI tool?

What approvals are needed for international travel?
```

Currently, employees may:

1. Search manually across multiple documents.
2. Misinterpret policy rules.
3. Ask HR or Finance repeatedly.
4. Receive inconsistent answers.
5. Make unsupported assumptions.
6. Miss required approvals or documentation.

The proposed assistant should help employees find policy-backed answers quickly and safely.

---

5. Product Goal

Build an **Enterprise Policy Assistant** that can:

1. Load multiple enterprise policy documents.
2. Chunk and index policy documents.
3. Retrieve relevant policy sections.
4. Classify the user query.
5. Search across multiple policy domains.
6. Grade whether retrieved context is relevant.
7. Rewrite the query if retrieval is weak.
8. Ask clarification if the user's question is ambiguous.
9. Generate a grounded answer with citations.
10. Review whether the final answer is supported by policy evidence.
11. Demonstrate LangGraph workflow patterns:
    - Sequential Pattern
    - Parallelization Pattern
    - Conditional Pattern
12. Demonstrate either:

- LangGraph pre-built ReAct agent, or
- Custom LangGraph ReAct-style agent.

---

6. Target Users

Primary User

Employee

A user who wants quick policy guidance.

Secondary User

HR / Finance / IT Support Team

Teams that repeatedly answer policy-related questions.

Optional User

Compliance or Risk Reviewer

A user who wants evidence-backed answers and safe handling of sensitive policy topics.

---

7. Dataset / Policy Corpus

Participants should use a local document corpus containing synthetic or trainer-provided enterprise policy files.

Recommended folder:

```
data/policies/
│
├── hr_leave_policy.md
├── travel_policy.md
├── reimbursement_policy.md
├── it_security_policy.md
└── ai_usage_policy.md
```

The trainer may provide these as `.md`, `.txt`, or `.pdf` files.

Recommended Policy Documents

Policy Document	Purpose
`hr_leave_policy.md`	Leave entitlement, carry forward, approvals, unpaid leave
`travel_policy.md`	Domestic travel, international travel, travel

Policy Document	Purpose
	approvals
`reimbursement_policy.md`	Meals, hotel, transport, receipts, claim limits
`it_security_policy.md`	Laptop usage, password rules, device security
`ai_usage_policy.md`	Public AI tool usage, customer data restrictions, approval process

8. Sample Policy Content Requirements

The policy corpus should include enough information to support realistic queries.

HR Leave Policy Should Cover

1. Annual leave entitlement.
2. Sick leave.
3. Carry-forward limits.
4. Leave approval workflow.
5. Unpaid leave.
6. Leave cancellation.

Travel Policy Should Cover

1. Domestic travel approval.
2. International travel approval.
3. Same-day travel.
4. Overnight travel.
5. Manager approval.
6. Client travel.

Reimbursement Policy Should Cover

1. Meal reimbursement.
2. Hotel reimbursement.
3. Receipt requirements.
4. Daily limits.
5. Non-reimbursable expenses.
6. Claim submission timeline.

IT Security Policy Should Cover

1. Company laptop usage.
2. Personal laptop restrictions.

3. Password security.
4. Public Wi-Fi usage.
5. Data storage rules.
6. Incident reporting.

AI Usage Policy Should Cover

1. Public AI tool restrictions.
2. Customer data handling.
3. Confidential data handling.
4. Approved AI tools.
5. AI use case approval.
6. Human review requirement.

---

9. Important Scope Boundary

This project must implement **Agentic RAG using LangGraph**.

Participants must not submit:

1. A simple LLM-only chatbot.
2. A single-prompt answer generator.
3. A basic one-step RAG pipeline only.
4. Hard-coded answers.
5. Answers without source evidence.
6. Answers from general model knowledge.

The assistant must answer using retrieved policy context.

---

10. Required Implementation Choices

Participants must choose one of the following options.

Choice 1: LangGraph Pre-built ReAct Agent

Participants use a LangGraph pre-built ReAct agent and provide custom tools.

This option is recommended for participants who want to focus on:

1. Tool design.
2. Retrieval tools.

3. Context grading tools.
4. Answer generation.
5. Policy-safe response generation.

Expected Capabilities

The pre-built agent should be able to call tools such as:

```
retrieve_hr_policy
retrieve_travel_policy
retrieve_reimbursement_policy
retrieve_it_security_policy
retrieve_ai_usage_policy
grade_retrieved_context
rewrite_query
generate_grounded_answer
```

Expected Architecture

```
User Question
   |
   v
Pre-built ReAct Agent
   |
   +--> Select Tool
   |
   +--> Retrieve Policy Context
   |
   +--> Grade Context
   |
   +--> Generate Final Answer
   |
   v
Grounded Answer with Citations
```

Choice 2: Custom LangGraph ReAct Agent

Participants manually build a LangGraph workflow using graph nodes, state, and conditional edges.

This option is recommended for participants who want to demonstrate deeper understanding of LangGraph.

Required Nodes

```
query_classifier_node
parallel_retrieval_node
context_grader_node
```

```
query_rewriter_node
answer_generator_node
reflection_node
final_response_node
```

Expected Architecture

```
START
  |
  v
query_classifier_node
  |
  v
parallel_retrieval_node
  |
  v
context_grader_node
  |
  +-- relevant context --> answer_generator_node
  |
  +-- weak context --> query_rewriter_node --> parallel_retrieval_node
  |
  +-- ambiguous query --> clarification_response_node
  |
  v
reflection_node
  |
  v
final_response_node
  |
  v
END
```

---

11. Required LangGraph Workflow Patterns

Participants must demonstrate the following three workflow patterns.

11.1 Sequential Pattern

The assistant must perform a multi-step workflow in sequence.

Required sequence:

```
User Question
   ↓
Query Classification
```

```
   ↓
Policy Retrieval
   ↓
Context Grading
   ↓
Answer Generation
   ↓
Answer Review / Reflection
   ↓
Final Response
```

Example

User asks:

```
Can I claim meals for same-day domestic business travel?
```

Expected sequential flow:

```
Classify as travel + reimbursement query
Retrieve travel policy and reimbursement policy
Grade context relevance
Generate grounded answer
Review whether answer is supported
Return final answer with citations
```

11.2 Parallelization Pattern

The assistant must search multiple policy domains in parallel or simulated parallel.

Required parallel retrieval branches:

```
HR Policy Retriever
Travel Policy Retriever
Reimbursement Policy Retriever
IT Security Policy Retriever
AI Usage Policy Retriever
```

The graph should merge retrieved results and rank the evidence.

Example

User asks:

```
What approvals are needed for international business travel?
```

Expected retrieval:

```
Travel policy branch retrieves international travel approval rules
Reimbursement policy branch retrieves expense claim requirements
HR policy branch may retrieve manager approval references
```

Then the system merges evidence and generates one answer.

11.3 Conditional Pattern

The assistant must branch based on context relevance, query type, or ambiguity.

Required conditional logic:

```
If context is relevant:
    Generate answer

If context is weak:
    Rewrite query and retrieve again

If query is ambiguous:
    Ask clarification

If query is unsupported or speculative:
    Return NOT_FOUND or policy-not-available response

If answer is not grounded:
    Revise or return insufficient-evidence response
```

Example

User asks:

```
Will my reimbursement definitely be approved?
```

Expected behavior:

```
The assistant should not guarantee approval.
It should say approval depends on policy conditions, documentation, and
manager/finance review.
```

---

12. Functional Requirements

FR-1: Document Loading

The application must load policy documents from a local folder.

Minimum supported formats:

```
.md
.txt
.pdf
```

Minimum requirement:

```
Load all files from data/policies/
```

Each document should retain metadata:

```
{
  "source_file": "",
  "policy_domain": "",
  "page_number": "",
  "chunk_id": ""
}
```

FR-2: Document Chunking

Participants must chunk documents before retrieval.

Recommended configuration:

```
chunk_size: 700 to 1000 characters
chunk_overlap: 100 to 150 characters
```

Each chunk must include:

```
{
  "chunk_id": "",
  "source_file": "",
  "policy_domain": "",
  "text": ""
}
```

FR-3: Embeddings and Vector Store

The application must generate embeddings and store chunks in a vector database.

Recommended vector stores:

```
Chroma
FAISS
InMemoryVectorStore
```

Recommended embedding options:

```
Hugging Face Sentence Transformers
FastEmbed
Open-source local embedding models
```

The vector store may be one combined index or separate indexes by policy domain.

FR-4: Query Classification

The assistant must classify the user query before answer generation.

Supported query types:

```
HR_LEAVE
TRAVEL
REIMBURSEMENT
IT_SECURITY
AI_USAGE
MULTI_POLICY
UNANSWERABLE
AMBIGUOUS
OTHER
```

Required output:

```
{
  "query_type": "",
  "required_policy_domains": [],
  "requires_parallel_retrieval": true,
  "requires_clarification": false,
  "reasoning_summary": ""
}
```

FR-5: Policy Retrieval

The assistant must retrieve policy chunks relevant to the user question.

Minimum requirement:

```
top_k = 3 to 5 chunks per relevant policy domain
```

The output should include:

```
{
  "policy_domain": "",
  "source_file": "",
  "chunk_id": "",
  "content": "",
  "relevance_score": 0.0
}
```

FR-6: Parallel Retrieval

For multi-policy questions, the assistant must retrieve from multiple policy domains.

Example:

```
Question: Can I use customer data in a public AI tool while working remotely?
```

Expected retrieval domains:

```
AI Usage Policy
IT Security Policy
Data Security / Privacy policy, if available
```

The graph must merge retrieved results before answer generation.

FR-7: Context Grading

The assistant must grade retrieved context before generating the final answer.

Supported grades:

```
HIGHLY_RELEVANT
PARTIALLY_RELEVANT
WEAK
NOT_RELEVANT
```

Required output:

```
{
  "overall_relevance": "",
  "relevant_chunks": [],
  "irrelevant_chunks": [],
  "missing_information": [],
```

```
  "decision": "ANSWER | REWRITE_QUERY | ASK_CLARIFICATION | NOT_FOUND"
}
```

FR-8: Query Rewriting

If retrieved context is weak, the assistant should rewrite the query and attempt retrieval again.

Example:

Original query:

```
Can I get money for food while travelling?
```

Rewritten query:

```
meal reimbursement rules for business travel same-day overnight domestic
travel receipts limits
```

Maximum retry requirement:

```
At most 1 query rewrite and retrieval retry
```

This prevents infinite loops.

FR-9: Clarification Handling

If the user's question is ambiguous, the assistant must ask a clarification question.

Example user query:

```
Can I claim this expense?
```

Expected response:

```
Please clarify the type of expense, travel type, date, and whether you have a
receipt.
```

The assistant should not guess.

FR-10: Grounded Answer Generation

The assistant must generate answers using only retrieved context.

Required answer format:

```
{
  "answer": "",
  "policy_basis": [],
  "sources": [],
  "answerability": "ANSWERED | PARTIALLY_ANSWERED | NOT_FOUND |
NEEDS_CLARIFICATION",
  "confidence": "HIGH | MEDIUM | LOW",
  "recommended_next_step": ""
}
```

Every important claim must map to a source.

FR-11: Source Citations

Each answer must include citations.

Minimum source fields:

```
{
  "source_file": "",
  "policy_domain": "",
  "chunk_id": "",
  "snippet": ""
}
```

For PDFs, page number should be included if available.

FR-12: Answer Reflection

The assistant must review the generated answer before final response.

Reflection should check:

1. Is the answer grounded in retrieved context?
2. Are citations present?
3. Is the answer overconfident?
4. Does the answer need clarification?
5. Did the assistant make unsupported claims?

Required output:

```json
{
  "is_grounded": true,
  "has_citations": true,
  "unsupported_claims": [],
  "needs_revision": false,
  "reflection_summary": ""
}
```

FR-13: Final Response

The final response should be clear, professional, and policy-safe.

It should include:

1. Direct answer.
2. Policy basis.
3. Source references.
4. Confidence.
5. Recommended next step.
6. Caveat when approval depends on review.

---

13. Tools to Implement

Participants must implement at least **6 tools** if using pre-built ReAct agent.

For custom graph implementations, these can be implemented as graph nodes or tools.

Recommended tools:

```
1. retrieve_hr_policy
2. retrieve_travel_policy
3. retrieve_reimbursement_policy
4. retrieve_it_security_policy
5. retrieve_ai_usage_policy
6. grade_context
7. rewrite_query
8. generate_grounded_answer
9. review_answer_grounding
10. ask_clarification
```

---

14. Suggested Graph State

For custom LangGraph implementation, participants may use a state object similar to:

```python
from typing import TypedDict, List, Dict, Optional

class PolicyAgentState(TypedDict):
    user_question: str
    query_type: str
    required_policy_domains: List[str]
    rewritten_query: Optional[str]
    retrieved_context: List[Dict]
    context_grade: Dict
    answer: Dict
    reflection: Dict
    retry_count: int
    final_response: str
```

---

15. Suggested Custom Graph Nodes

Node 1: Query Classifier

Input:

`user_question`

Output:

`query_type, required_policy_domains, clarification flag`

Node 2: Parallel Retrieval

Input:

`question, required_policy_domains`

Output:

`retrieved chunks from relevant policy domains`

Node 3: Context Grader

Input:

`question, retrieved_context`

Output:

`relevance grade and decision`

Node 4: Query Rewriter

Input:

`question, missing_information, weak context`

Output:

`rewritten_query`

Node 5: Answer Generator

Input:

`question, retrieved_context`

Output:

`grounded answer with citations`

Node 6: Reflection Node

Input:

`answer, retrieved_context`

Output:

`grounding check and revision recommendation`

Node 7: Final Response Node

Input:

`answer, reflection`

Output:

`final user-facing response`

---

16. Required Conditional Edges

Participants must implement conditional edges similar to:

```
After query_classifier_node:
    if requires_clarification == true -> clarification_response_node
    else -> parallel_retrieval_node

After context_grader_node:
    if decision == ANSWER -> answer_generator_node
    if decision == REWRITE_QUERY and retry_count < 1 -> query_rewriter_node
    if decision == ASK_CLARIFICATION -> clarification_response_node
    if decision == NOT_FOUND -> final_response_node

After reflection_node:
    if needs_revision == true -> answer_generator_node or final_response_node
with caveat
    else -> final_response_node
```

---

17. Sample User Questions

Participants must test at least 8 questions.

```
1. How many annual leave days can an employee carry forward?
2. Can I claim meals for same-day domestic business travel?
3. What documents are needed for hotel reimbursement?
4. Can I use my personal laptop for office work?
5. What approvals are needed for international travel?
6. Can customer data be uploaded to a public AI tool?
7. Will my reimbursement definitely be approved?
8. What should I do if the policy does not mention my scenario?
```

---

18. Expected Behavior for Sample Questions

Question 1

```
How many annual leave days can an employee carry forward?
```

Expected behavior:

```
Retrieve HR leave policy.
Answer based on carry-forward rule.
Cite HR policy source.
```

Question 2

```
Can I claim meals for same-day domestic business travel?
```

Expected behavior:

```
Retrieve travel policy and reimbursement policy.
Explain same-day travel eligibility and meal reimbursement limit if provided.
Mention receipt requirement if policy says so.
Do not invent amounts if not present.
```

Question 3

```
What documents are needed for hotel reimbursement?
```

Expected behavior:

```
Retrieve reimbursement policy.
Answer with required documents such as hotel bill, receipt, approval, travel
request, if present in policy.
```

Question 4

```
Can I use my personal laptop for office work?
```

Expected behavior:

```
Retrieve IT security policy.
Answer according to device policy.
Mention security approval or restrictions if present.
```

Question 5

```
What approvals are needed for international travel?
```

Expected behavior:

```
Retrieve travel policy, reimbursement policy, and possibly manager approval
policy.
Return approval chain with citations.
```

Question 6

```
Can customer data be uploaded to a public AI tool?
```

Expected behavior:

```
Retrieve AI usage policy and IT security policy.
Answer safely.
```

```
Usually expected: do not upload customer/confidential data to public AI tools
unless approved.
```

Question 7

```
Will my reimbursement definitely be approved?
```

Expected behavior:

```
Do not guarantee approval.
Explain approval depends on policy eligibility, receipts, limits, and review.
Return PARTIALLY_ANSWERED or policy-safe answer.
```

Question 8

```
What should I do if the policy does not mention my scenario?
```

Expected behavior:

```
Retrieve relevant general policy if available.
Recommend contacting HR/Finance/IT policy owner or manager.
Do not invent missing policy.
```

---

19. Expected Final Output Example

User question:

```
Can I claim meals for same-day domestic business travel?
```

Expected output shape:


```
{
  "answer": "Based on the retrieved travel and reimbursement policy sections,
same-day domestic business travel may be eligible for meal reimbursement if
the trip meets the policy conditions and valid receipts are submitted. The
claim should follow the reimbursement limits defined in the policy. If the
policy does not specify the exact amount for your scenario, finance review is
required.",
  "policy_basis": [
    "Travel policy mentions same-day business travel eligibility.",
    "Reimbursement policy mentions meal reimbursement and receipt
requirements."
  ],
  "sources": [
    {
      "source_file": "travel_policy.md",
      "policy_domain": "TRAVEL",
```

```
      "chunk_id": "travel_chunk_003",
      "snippet": "Same-day domestic business travel must be approved by the
reporting manager..."
    },
    {
      "source_file": "reimbursement_policy.md",
      "policy_domain": "REIMBURSEMENT",
      "chunk_id": "reimbursement_chunk_005",
      "snippet": "Meal reimbursement requires valid receipts and must stay
within the daily policy limit..."
    }
  ],
  "answerability": "ANSWERED",
  "confidence": "HIGH",
  "recommended_next_step": "Submit the claim with receipts and manager-
approved travel details."
}
```


---

20. Non-Functional Requirements

NFR-1: Reliability

The application should handle:

1. Empty user questions.
2. Missing policy files.
3. Empty retrieval results.
4. Weak context.
5. Failed LLM calls.
6. Invalid JSON from model.
7. Infinite retry prevention.
8. Unsupported questions.

NFR-2: Security and Privacy

Participants must follow these rules:

1. Do not hard-code API keys.
2. Do not upload confidential company documents.
3. Use only the provided or synthetic policy dataset.
4. Do not invent policy rules.
5. Do not provide legal/HR/financial approval guarantees.

6. Warn that final policy decisions require human review where applicable.

NFR-3: Maintainability

Suggested file responsibilities:

File	Responsibility
`app.py`	Main entry point
`config.py`	Environment variables
`loaders.py`	Document loading
`chunking.py`	Text splitting
`retrievers.py`	Policy retrievers
`tools.py`	Tool definitions
`graph.py`	Custom LangGraph workflow
`prebuilt_agent.py`	Pre-built ReAct implementation
`prompts.py`	Prompt templates
`output_parser.py`	JSON parsing
`README.md`	Setup and usage

21. Suggested Folder Structure

```
enterprise_policy_agentic_rag/
│
├── app.py
├── config.py
├── loaders.py
├── chunking.py
├── retrievers.py
├── tools.py
├── graph.py
├── prebuilt_agent.py
├── prompts.py
├── output_parser.py
├── requirements.txt
├── .env.example
├── README.md
│
├── data/
│   └── policies/
│       ├── hr_leave_policy.md
│       ├── travel_policy.md
│       ├── reimbursement_policy.md
│       ├── it_security_policy.md
```

```
│       └── ai_usage_policy.md
│
├── vector_store/
│
└── outputs/
    ├── sample_run_outputs.md
    └── evaluation_results.json
```

---

22. Recommended Technology Stack

Minimum:

```
Python
LangGraph
LangChain
langchain-groq
Vector store
Embedding model
python-dotenv
```

Recommended packages:

```
langgraph
langchain
langchain-core
langchain-community
langchain-groq
langchain-chroma
chromadb
sentence-transformers
python-dotenv
pydantic
```

Optional:

```
streamlit
gradio
faiss-cpu
rich
tabulate
```

---

23. Environment Variables

`.env.example`:

```
GROQ_API_KEY=
GROQ_MODEL=llama-3.3-70b-versatile
```

```
EMBEDDING_MODEL=sentence-transformers/all-MiniLM-L6-v2
POLICY_DATA_PATH=data/policies
VECTOR_STORE_PATH=vector_store
CHUNK_SIZE=900
CHUNK_OVERLAP=120
TOP_K=4
```

---

24. Deliverables

Participants must submit:

```
1. Source code
2. requirements.txt
3. .env.example
4. README.md
5. Policy dataset used
6. Sample outputs for at least 8 test questions
7. Explanation of implementation choice:
   - Pre-built ReAct agent, or
   - Custom LangGraph agent
8. Explanation of sequential pattern
9. Explanation of parallelization pattern
10. Explanation of conditional pattern
11. Explanation of hallucination-control strategy
12. Known limitations and future improvements
```

---

25. Evaluation Rubric

Total Marks: 50

25.1 Agentic RAG Pipeline — 8 Marks

Criteria	Marks
Loads and indexes policy documents	1.5
Uses embeddings and vector store	1.5
Retrieves relevant chunks	1.5
Generates grounded answer using retrieved context	1.5
Provides citations	1
Handles insufficient context safely	1

25.2 LangGraph Implementation — 8 Marks

Criteria	Marks
Uses LangGraph correctly	2
Implements pre-built ReAct or custom graph	2
Uses graph state or agent state meaningfully	1
Uses nodes/tools appropriately	1
Demonstrates multi-step execution	1
Documents implementation choice	1

25.3 Sequential Pattern — 5 Marks

Criteria	Marks
Implements ordered workflow	2
Includes query classification	1
Includes retrieval before generation	1
Includes final review/reflection	1

25.4 Parallelization Pattern — 6 Marks

Criteria	Marks
Retrieves from multiple policy domains	2
Merges evidence from parallel branches	1.5
Handles multi-policy questions	1.5
Explains parallel design	1

25.5 Conditional Pattern — 6 Marks

Criteria	Marks
Branches based on query type	1
Branches based on context relevance	1.5
Implements query rewrite path	1

Criteria	Marks
Implements clarification path	1
Handles not-found/unsupported questions	1
Prevents infinite retry loops	0.5

25.6 Tool Design and Prompt Quality — 5 Marks

Criteria	Marks
Defines useful tools or graph nodes	1.5
Uses clear system prompts	1
Includes grounding rules	1
Produces structured outputs	1
Avoids unsupported policy claims	0.5

25.7 Testing and Output Quality — 5 Marks

Criteria	Marks
Tests at least 8 required questions	2
Answers are policy-grounded	1
Citations are clear	1
Responses are professional and useful	1

25.8 Documentation and Submission Quality and use of AI tools to generate code — 7 Marks

Criteria	Marks
Clear README	1.5
Setup and run instructions	1
Explains dataset and indexing	1
Explains graph design	1
Explains sequential, parallel, and conditional patterns	1.5
Lists limitations and future improvements	1

26. Final Marks Summary

Area	Marks
Agentic RAG Pipeline	8
LangGraph Implementation	8
Sequential Pattern	5
Parallelization Pattern	6
Conditional Pattern	6
Tool Design and Prompt Quality	5
Testing and Output Quality	5
Documentation and Submission Quality and use of AI tools to generate code	7
Total	**50**

27. Grade Bands

Score	Interpretation
45–50	Excellent, industry-grade LangGraph Agentic RAG implementation
38–44	Strong implementation with minor gaps
30–37	Acceptable working implementation
25–29	Partially complete
Below 25	Insufficient

28. Minimum Passing Criteria

A participant must demonstrate:

1. Working LangGraph implementation.
2. RAG over policy documents.
3. Groq integration.
4. Sequential workflow.
5. At least one parallel retrieval workflow.
6. At least one conditional branch.
7. Grounded answer with source citations.
8. Handling of unsupported or ambiguous question.
9. README and sample outputs.

29. Strong Submission Criteria

A strong submission should include:

1. Clean graph state.
2. Clear graph nodes.
3. Query classification.
4. Parallel policy retrieval.
5. Context grading.
6. Query rewriting.
7. Answer reflection.
8. Policy-safe answers.
9. Good citations.
10. Clear documentation.

---

30. Excellent Submission Criteria

An excellent submission should include:

1. Both pre-built ReAct and custom LangGraph implementation.
2. Strong conditional routing.
3. Relevance grading with structured scores.
4. One retry loop with query rewriting.
5. Clarification path for ambiguous queries.
6. Answer grounding review.
7. Clean UI or CLI.
8. Modular code.
9. Robust error handling.
10. Excellent README with architecture diagram.

---

31. Common Mistakes and Suggested Deductions

Mistake	Suggested Deduction
Does not use LangGraph	10–15
No RAG implementation	10–15
Simple chatbot only	10–20
No vector store or embeddings	6–10
No citations	4–6

Mistake	Suggested Deduction
No sequential workflow	3–5
No parallel retrieval	4–6
No conditional branching	4–6
No query rewriting or clarification path	2–4
Answers from general knowledge	5–10
Hallucinates policy rules	5–10
No README	3–5
Cannot run project	20–30

32. Final Instruction to Participants

You are building an enterprise-grade policy assistant using LangGraph Agentic RAG.

The focus is not only on answering policy questions. The focus is on designing a graph-based agent workflow that can:

1. Retrieve policy context.
2. Decide whether the context is sufficient.
3. Branch based on query type and evidence quality.
4. Search across multiple policies.
5. Rewrite poor queries.
6. Ask clarification when required.
7. Generate grounded answers with citations.
8. Reflect before final response.

Your assistant must use the policy documents as the source of truth. Do not invent policy rules.